Teacher guide

MapReduce

Counting words with a team of machines

Each machine reads only part of the text. How do they still count every word exactly, none missed or counted twice?

By the end you can

Count every word of a text exactly, with machines that each read only part of it.

- Say why the text is cut into pieces and read in parallel, and what that buys.
- Explain why every copy of a word must reach the same counter, and how a hash does it without asking anyone.
- Say why a crashed or slow reading task can simply run again, and which finished work must re-run.
- Name what the shuffle costs, and how adding up repeats early cuts it.

Open it

The file mapreduce.html, in any browser, offline.

4 stories, about 7 min 55 s of play at normal speed. Each question below names its story and step; the timeline and the share link go straight there.

The plan

Story	Opens at	Play	Questions	Talk
Everything works	the start	2 min	1	16 min
One machine crashes · what if a machine crashes halfway through	step 3 · Map	1 min 55 s	1	4 min
One machine is slow · what if one machine is much slower than the rest	step 3 · Map	1 min 55 s	1	4 min
One word is very common · what if one word is far more common than the rest	the start	2 min 5 s	1	5 min
In all		**7 min 55 s**	**4**	**29 min**

Play times are measured from each story's run at normal speed, with repeated steps sped through as the page plays them; a story that starts where it leaves the first counts from there. Talk times are the lesson's suggestion for each chapter's prompt. Pauses for questions come on top.

Running it in a room

- Press P for presenter mode: big type and high contrast for a projector. A clicker steps through it: → or Page Down goes on, ← or Page Up goes back.
- Presenter view (in the toolbar while presenting) opens the words on your laptop: what is on screen, what comes next, a warning before each question, the answer, what each wrong answer means, and the prompt for each chapter.
- Switch story from the presenter view or the scenarios strip. Each story starts where it leaves the first, so the shared setup is not replayed.

- To open a story or step straight away, use Share → Link to this moment, or add #scenario-<id>&step-<n> to the address.

Wrong ideas to listen for

“Each machine counts its piece, and one machine adds it all up at the end.”	Then one machine would add up every slip. Instead each word's slips meet on one counter, so the adding is shared too.
“Spreading the slips evenly over the counters is best.”	Copies of one word would land on different counters, and none could give its full count.
“A crashed machine's words are lost, so the count comes out short.”	Its piece is still in the input. Reading it again gives the same slips.
“One failure restarts the whole job.”	Only the failed machine's tasks run again.
“The job has to wait for its slowest machine.”	A backup copy of the slow piece on a free machine gives the same slips. The first copy to finish wins.
“Skipping a slow piece is a fair price for finishing on time.”	Its words would never be counted. MapReduce never trades a correct count for speed.

The picture: a library counted by a team

the coordinator handing out shelves	→	the master
one shelf of books	→	one piece of the text (a split)
a helper tallying their shelf	→	a reading machine (a mapper)
a tally slip “fox: 1”	→	a record `(fox, 1)`: the word, and the number 1
“all slips for one word go to one person”	→	a fixed rule on the word's letters (a hash) that picks one counting machine
the person adding them up	→	a counting machine (a reducer)
the noticeboard	→	the output: a folder with one file per counting machine

Where the picture breaks: Helpers would talk to agree who counts which word. Machines never talk about it: each one runs the same rule on the word and gets the same answer.

Words

master · the coordinator
The machine that hands out pieces of work, keeps track of them, and notices when a machine fails.

split · one piece of the text
One piece of the input, read by one machine, like one shelf of a library.

mapper · a reading machine
A machine running map: it reads one piece and writes a slip like (fox, 1) for every word.

hash · a word's fixed number
A function that turns a word into a big number. The same word always gives the same number.

shuffle · sending each slip to its counter
The step where every slip travels to the machine that adds up its word.

reducer · a counting machine
A machine running reduce: it adds up all the slips for its share of the words.

straggler · a slow machine
A machine much slower than the rest, so everyone ends up waiting for it.

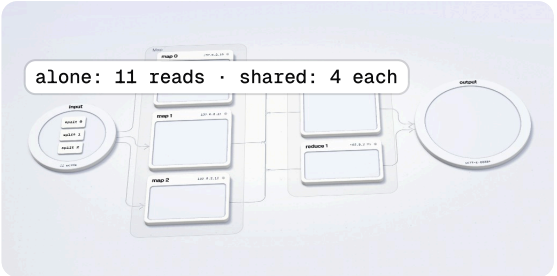
combiner · adding up early
Code that adds up repeats on the reading machine itself, before any slip is sent.

Story 1 · **Everything works**

Every machine stays healthy: the whole job, from cutting up the text to the final counts.

① Split · 10 s of play

Think of counting every word in a library with a team. One person would take ages, so a coordinator, the master, gives each helper one shelf: a split, one piece of the text.



End of Split · step 2

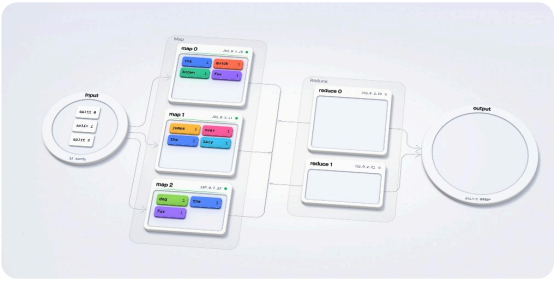
> Ask the room · 3 min
>
> **Before it runs: if three people share the counting of one library, what could go wrong with the total?**
>
> **Listen for** A word counted twice, or missed, where two pieces meet. Here every word sits in exactly one piece, so neither can happen.

Step 1 Count the words in “the quick brown fox jumps over the lazy dog the fox”: 11 in all. One machine alone would read all 11, one after another. Shared by 3 machines, none reads more than 4.

Step 2 The master cuts the text into 3 pieces of about 4 words, one per machine. The pieces are `• split 0` “the quick brown fox”, `• split 1` “jumps over the lazy” and `• split 2` “dog the fox”. Each can be read without the others, so all 3 machines start at once.

② Map · 20 s of play

Each helper tallies their own shelf: each reading machine, a mapper, writes a slip like `(word, 1)`, meaning “this word, seen once”, for every word. Nobody waits for anybody.



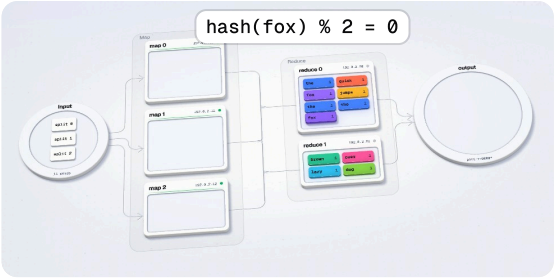
End of Map · step 6

> Ask the room · 3 min
>
> **Why can every reading machine start at once, without waiting for the others?**
>
> **Listen for** Map needs only its own piece, and nobody adds anything yet. Listen for “one machine adds it all up at the end”: the shuffle answers that idea next.

Step 4 `• map 0` writes a slip for each of its 4 words, like `(the, 1)`. That is all map does: label each word with a 1. The adding comes later, so nobody waits.

③ Shuffle · 45 s of play

In the shuffle, every slip for one word must reach the same counting machine. A hash turns the word into a number. Divide it by the number of counters: the remainder, from 0 up to one less, names the counter.



End of Shuffle · step 17

Ask the room · 5 min

Why not deal the slips out evenly, like cards, so every counter gets the same amount?

Listen for Copies of one word would land on different counters, and no counter could give that word's full count. The hash sends every copy to one counter without any machine asking another.

Step 7 `the` goes to `• reduce 0`: its hash, 3 020 861 980, divided by 2 leaves remainder 0. Taking turns would scatter one word's slips over several counters, and counters never compare notes, so none would hold the full count.

Q1 Your turn Everything works · at step 13

The first `the` went to reduce 0. Click the counting machine that should get this one, from `• map 1`.

On the page you click your answer in the world; here, tick it.

- [x] A **reduce 0**
- [ ] B reduce 1

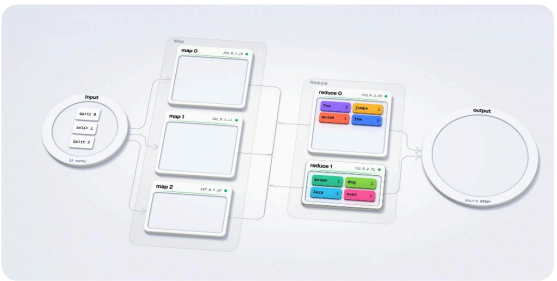
  Now `the`'s slips sit on two counters. Each adds up only what it holds, so neither can give the full count. Wrong idea: “Spreading the slips evenly over the counters is best.”

Why `the` always hashes to the same number, so every copy of it meets on reduce 0.

Step 13 `the` from `• map 1` goes to `• reduce 0` too: the same word always gives the same hash. The hash looks only at the word's letters, so every copy meets on one counter, whichever reader sent it.

④ Reduce · 20 s of play

Each counting machine, a reducer, sorts its slips so equal words sit together, then adds up each group into one total.



End of Reduce · step 21

Ask the room · 2 min

Why sort the slips before adding them up, when a dictionary would do?

Listen for Sorted, equal words sit together, so one pass adds them up even when the slips don't fit in memory.

Step 18 `• reduce 0` sorts its 7 slips so equal words sit together. Sorted, each word's slips sit together, so one pass adds them up, even when they don't fit in memory.

Step 19 `• reduce 0` adds up each group: `fox` = 2 and `the` = 3. The hash sent every copy of these words here and nowhere else, so each total is the full count. The rest stay at 1: `jumps` and `quick`.

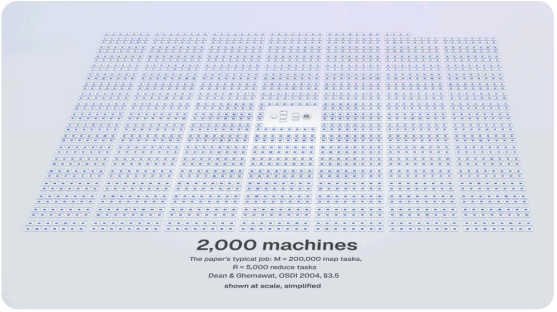
⑤ Output · 30 s of play

Each counter posts its totals to the shared output, like a list on a noticeboard. Together they hold exactly one count for every different word.

Ask the room · 3 min

Point at any total in the output. How do we know no other counter holds part of it?

Listen for The hash sent every copy of that word to one counter, so its total is the full count. Every what-if tests exactly this.



End of Output · step 25

Step 24 Done: all 11 words counted, 8 different ones, and no machine read the whole text. Every count is exact because every copy of a word met on one counter; the what-ifs test that with a crash, a slow disk and a common word.

Step 25 The same job at the paper's scale: 200,000 map tasks, 5,000 reduce tasks, 2,000 machines. Shown simplified. These are the numbers from Google's 2004 MapReduce paper: each machine runs many tasks in turn, and every copy of a word still meets on one counter.

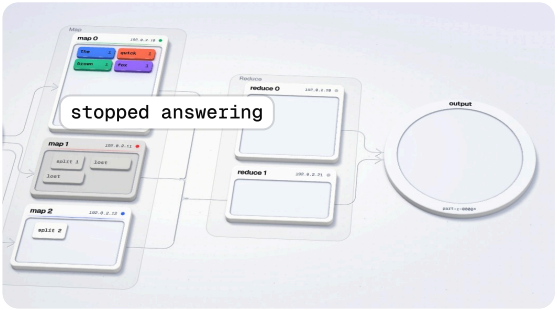
Story 2 · One machine crashes

What if a machine crashes halfway through?

One reading machine crashes halfway through its piece, and the master has that piece read again.

It starts where it leaves the first story: step 3 of 27.

Takeaway A machine died mid-task and the answer did not change: its piece was still in the shared input, so the master had it read again.



Map · step 6

② Map · 35 s of play

Each reading machine, a mapper, writes slips like `(word, 1)` to its own disk until the counters fetch them. If it crashes first, they die with it, so its piece is read again, even if it had finished.

Ask the room · 4 min

A machine just crashed. Before the question on screen, ask the room: is its part of the count gone for good?

Listen for Its text is still in the shared input, so the master hands the piece out again. Listen for “the count comes out short” and “the whole job starts over”.

Step 6 `• map 1` crashes after 2 of 4 words; its 2 slips are lost with it. Its slips were on its own disk, so they died with it: the grey slips marked “lost” no longer count.

Q2 **Predict** One machine crashes · before step 8

map 1 crashed after 2 of its 4 words. What happens to its piece?

- [ ] A The count comes out 2 words short
  Only if nobody read the piece again. Its text is still in the shared input, so the master hands it out once more. Wrong idea: “A crashed machine's words are lost, so the count comes out short.”
- [x] B **The master has the piece read again**
- [ ] C The whole job starts over
  Nothing else was lost: the other machines still hold their slips. Only the dead machine's piece runs again. Wrong idea: “One failure restarts the whole job.”

Why The text is still in the shared input, and reading it again gives the same slips, so redoing just that piece is safe.

Step 8 `• map 0` is free, so the master hands it split 1: **the text is still in the input.** The input is stored apart from the machines that read it, so a crash loses work, never text.

---

③ Shuffle · 45 s of play

In the shuffle, every slip for one word must reach the same counting machine. A hash turns the word into a number. Divide it by the number of counters: the remainder, from 0 up to one less, names the counter.

Ask the room · 5 min

Why not deal the slips out evenly, like cards, so every counter gets the same amount?

Listen for Copies of one word would land on different counters, and no counter could give that word's full count. The hash sends every copy to one counter without any machine asking another.

Step 16 `the` from `• map 0` goes to `• reduce 0` too: **the same word always gives the same hash.** The hash looks only at the word's letters, so every copy meets on one counter, whichever reader sent it.

---

⑤ Output · 15 s of play

Each counter posts its totals to the shared output, like a list on a noticeboard. Together they hold exactly one count for every different word.

Ask the room · 3 min

Point at any total in the output. How do we know no other counter holds part of it?

Listen for The hash sent every copy of that word to one counter, so its total is the full count. Every what-if tests exactly this.

Step 27 **Done: all 11 words counted exactly once, despite the crash.** The lost piece was read again from the input, and the dead copy's slips never reached a counter.

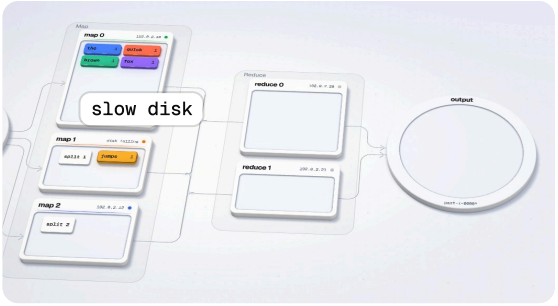
Story 3 · One machine is slow

What if one machine is much slower than the rest?

One reading machine has a failing disk, so the master races a backup copy of its piece on a free machine.

It starts where it leaves the first story: step 3 of 27.

Takeaway One slow disk could have held up the whole job. A backup copy of its piece finished first, and the answer is the same.



Map · step 5

② Map · 40 s of play

Each reading machine, a mapper, writes a slip for every word. One has a failing disk, a straggler, and the job waits for it unless the master runs a backup copy of its piece.

Ask the room · 4 min

One machine is far behind. Should the job wait for it, skip it, or do something else?

Listen for A backup copy of the slow piece runs on a free machine, and the first copy to finish wins. Skipping it would leave its words uncounted.

Step 5 `• map 1` 's failing disk keeps retrying reads: it has written 1 of 4 slips. Counting can't finish until every piece has been read, so one slow machine holds up the whole job.

Q3 **Predict** One machine is slow · before step 7

map 1 has written 1 of its 4 slips, and the other machines are done. What does the master do?

- [ ] A Wait: the job cannot finish without its piece
  Waiting works, but then the whole job takes as long as the slowest disk. A free machine could read the same piece. Wrong idea: “The job has to wait for its slowest machine.”
- [x] B **Start a copy of its piece on a free machine**
- [ ] C Skip its words to finish on time
  Then those words would never be counted, and the answer would be wrong. Speed never buys a wrong count here. Wrong idea: “Skipping a slow piece is a fair price for finishing on time.”

Why Reading a piece twice gives the same slips, so a backup copy is safe, and the master keeps whichever finishes first.

Step 7 `• map 0` is free, so the master starts a backup copy of split 1 on it. Two copies of one piece write the same slips, so whichever finishes first can be kept.

③ Shuffle · 45 s of play

In the shuffle, every slip for one word must reach the same counting machine. A hash turns the word into a number. Divide it by the number of counters: the remainder, from 0 up to one less, names the counter.

Ask the room · 5 min

Why not deal the slips out evenly, like cards, so every counter gets the same amount?

Listen for Copies of one word would land on different counters, and no counter could give that word's full count. The hash sends every copy to one counter without any machine asking another.

Step 16 `the` from `• map 0` goes to `• reduce 0` too: the same word always gives the same hash. The hash looks only at the word's letters, so every copy meets on one counter, whichever reader sent it.

⑤ Output · 15 s of play

Each counter posts its totals to the shared output, like a list on a noticeboard. Together they hold exactly one count for every different word.

Ask the room · 3 min

Point at any total in the output. How do we know no other counter holds part of it?

Listen for The hash sent every copy of that word to one counter, so its total is the full count. Every what-if tests exactly this.

Step 27 Done: all 11 words counted exactly once, despite the slow disk. The backup's slips counted and the slow copy's were thrown away, so no word was counted twice.

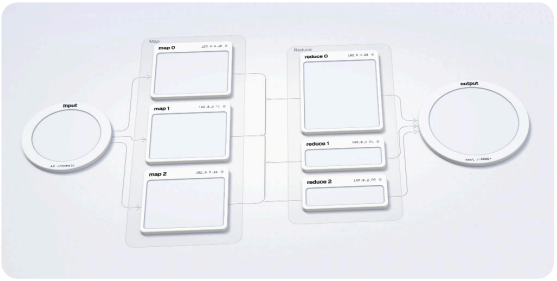
Story 4 · One word is very common

What if one word is far more common than the rest?

A web log where almost every line is the status code 200, so one counting machine gets most of the work.

29 steps.

Takeaway Every copy of a word goes to one counter: counts stay exact, but a very common word loads that machine. Adding up repeats on each reader first shrinks the load.



Split · step 1

① Split · 10 s of play

The same counting job on a web log. Each line is one website request, and its status code is a number saying how it went (200 OK, 404 not found). The master gives each machine one split of the log.

Ask the room · 3 min

Before it runs: if three people share the counting of one library, what could go wrong with the total?

Listen for A word counted twice, or missed, where two pieces meet. Here every word sits in exactly one piece, so neither can happen.

Step 1 Count each status code in a web log of 14 requests. It is the same job as counting words, with codes for words: shared by 3 machines, none reads more than 5.

Step 2 The master cuts the log into 3 pieces of about 5 lines, one per machine. The pieces are `• split 0` “200 200 404 200 200”, `• split 1` “301 200 200 500 200” and `• split 2` “200 200 304 200”. Each can be read without the others, so all 3 machines start at once.

② Map · 20 s of play

Each reading machine, a mapper, writes a slip like `(code, 1)` for every line of its piece. Nobody waits for anybody.

Ask the room · 3 min

Why can every reading machine start at once, without waiting for the others?

Listen for Map needs only its own piece, and nobody adds anything yet. Listen for “one machine adds it all up at the end”: the shuffle answers that idea next.

Step 4 `• map 0` writes a slip for each of its 5 lines, like `(200, 1)`. Map labels each line with a 1, even repeats: every `(200, 1)` travels alone unless its reader adds them up first.

③ Shuffle · 50 s of play

In the shuffle, each slip goes to the counting machine its hash picks. `200` fills 10 of the 14 lines, so one machine receives every one of those slips.

Ask the room · 5 min

Almost every line is the same code. Where will all those slips go, and what does that do to one machine?

Listen for Every copy goes to the one counter its hash picks: the count stays exact, but that counter does most of the work. Adding up repeats first shrinks it.

Step 7 `200` goes to `• reduce 0`: its hash, 3 286 718 301, divided by 3 leaves remainder 0. Taking turns would scatter one code's slips over several counters, and counters never compare notes, so none would hold the full count.

Q4 Predict One word is very common · before step 8

The first "200" went to reduce 0. Where does this "200", from map 0, go?

- [ ] A Reduce 1: spread the slips evenly
  Even spreading would split `200`'s slips between reduce 0 and reduce 1, and neither could give its full count. Wrong idea: “Spreading the slips evenly over the counters is best.”
- [x] B **Reduce 0, to join the first "200"**
- [ ] C Nowhere yet: one machine counts everything at the end
  Then one machine would add up every slip, the very work we shared out. Here each counter adds up only its own share. Wrong idea: “Each machine counts its piece, and one machine adds it all up at the end.”

Why hash(200) is the same number every time, so its remainder is 0 every time: every "200" meets on reduce 0.

Step 8 `200` from `• map 0` goes to `• reduce 0` too: the same code always gives the same hash. The hash looks only at the code's characters, so every copy meets on one counter, whichever reader sent it.

④ Reduce · 25 s of play

Each counting machine, a reducer, sorts its slips so equal codes sit together, then adds up each group into one total.

Ask the room · 2 min

Why sort the slips before adding them up, when a dictionary would do?

Listen for Sorted, equal words sit together, so one pass adds them up even when the slips don't fit in memory.

Step 21 `• reduce 0` sorts its 12 slips so equal codes sit together. Sorted, each code's slips sit together, so one pass adds them up, even when they don't fit in memory.

Step 22 `• reduce 0` adds up each group: `200` = 10. The hash sent every copy of these codes here and nowhere else, so each total is the full count. The rest stay at 1: `301` and `404`.

⑤ Output · 20 s of play

Each counter posts its totals to the shared output, like a list on a noticeboard. Together they hold exactly one count for every different status code.

Ask the room · 3 min

Point at any total in the output. How do we know no other counter holds part of it?

Listen for The hash sent every copy of that word to one counter, so its total is the full count. Every what-if tests exactly this.

Step 28 Done. `• reduce 0` got 12 of 14 slips, 2.6× a fair share (14 ÷ 3 counters). `200` fills 10 of the 14 lines, and every raw `200` slip must meet on one counter; other codes can hash there too.

Step 29 If each reader sent one slip per different code, 7 slips would travel, 5 to `• reduce 0`. Each slip would carry its code's count instead of a 1, so the busy counter's load falls and counts stay exact.

Check yourself

New questions, at other settings than the page's: the rules decide, not memory.

Q5 Check yourself Everything works · after the run

Suppose a new job with 3 counting machines instead of 2. Which one gets every `the` slip?

- [ ] A reduce 0
- [x] B **reduce 1**
- [ ] C reduce 2

Why hash(`the`) is the same on every machine. Divided by 3, it leaves remainder 1: reduce 1.

Q6 **Check yourself** Everything works · after the run

Suppose a new job with 4 reading machines instead of 3. How many words does the busiest one read?

- [ ] A 2
- [x] B **3**
- [ ] C 11

Why 11 words shared by 4 machines: the biggest piece has 3.

Q7 **Check yourself** Everything works · after the run

Same 3 reading machines and 2 counting machines: add one more `the` to the text. How many totals would the output hold?

- [x] A **8**
- [ ] B 9
- [ ] C 12

Why The output holds one total per different word. One more `the` changes its count, not how many totals there are: 8.

What you saw

1. The text was cut into pieces, and each machine read one.
2. Each reader wrote a slip for every word, like (fox, 1).
3. A hash sent every copy of a word to the same counting machine.
4. Each counter added up its words; together they hold every count.

Every count is exact because every copy of a word met on one counter. The what-ifs test it with a crash, a slow disk and one very common word.

Answers

The same answers as above, for a quick check of a stack of worksheets.

Q1 **A reduce 0** · Everything works · at step 13
`the` always hashes to the same number, so every copy of it meets on reduce 0.

Q2 **B The master has the piece read again** · One machine crashes · before step 8
The text is still in the shared input, and reading it again gives the same slips, so redoing just that piece is safe.

Q3 **B Start a copy of its piece on a free machine** · One machine is slow · before step 7
Reading a piece twice gives the same slips, so a backup copy is safe, and the master keeps whichever finishes first.

Q4 **B Reduce 0, to join the first "200"** · One word is very common · before step 8
hash(200) is the same number every time, so its remainder is 0 every time: every "200" meets on reduce 0.

Q5 **B reduce 1** · Everything works · after the run
hash(`the`) is the same on every machine. Divided by 3, it leaves remainder 1: reduce 1.

Q6 **B 3** · Everything works · after the run
11 words shared by 4 machines: the biggest piece has 3.

Q7 **A 8** · Everything works · after the run
The output holds one total per different word. One more `the` changes its count, not how many totals there are: 8.

Made by the page from its own run: every number above comes from the simulation. Source: Dean & Ghemawat, OSDI 2004.